ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N. °1

Consolidación de reportes de ventas

Tópico 1 del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante: Encalada Oña Orlin Emanuel – usuario de Github: naaairol

Docente: Suarez Cobos Christian Gonzalo

Período académico: 2026-B

Fecha de entrega: 19/10/2026

Repositorio: https://github.com/naaairol/IDSD422-lab1-lectura-datos.git

1. Introducción

La empresa RetailAndina S.A. es una cadena ecuatoriana con presencia en las principales ciudades del Ecuador. Contando con varias sucursales, siendo estas: Quito Norte, Quito Sur, Guayaquil Centro, Guayaquil Norte y Cuenca. La empresa recibe los reportes de las ventas desde tres sistemas diferentes, cada uno en un formato diferente. Desde el ERP de sucursales recibe el reporte en formato CSV, desde la plataforma de fidelización lo recibe en un formato JSON y, finalmente, desde el sistema de finanzas se entregan los datos mediante un archivo tipo Excel; XLSX. El reto radica en verificar si los datos provenientes de los distintos sistemas es la misma, independientemente del formato en que se encuentran.

2. Objetivos

Objetivo general: Analizar el reporte de la semana 39 de la empresa RetailAndina S.A., mediante el uso de lenguajes de programación y librerías específicas, para verificar que la información provista es consistente, así como para determinar ciertos indicadores comerciales relevantes para la empresa.

Objetivos específicos:

- Cargar los tres archivos de datos utilizando scripts escritos en Python.
- Verificar las dimensiones y variables de los tres dataframes.
- Calcular indicadores comerciales relevantes para RetailAndina S.A.

3. Desarrollo de la práctica

3.1. Fork al repositorio principal

Lo primero que realicé fue un fork del repositorio principal provisto por el docente (Suárez, 2026). De esta forma siendo posible trabajar sobre una versión propia del repositorio, posibilitando el control de versiones.

3.2. Preparación del entorno

Para el desarrollo de la práctica y ejecución de scripts, no trabajé de manera local. Todo lo realicé en la nube, mediante Google Colab. Por lo que, la preparación del entorno únicamente fue la creación de un nuevo cuaderno, en la plataforma previamente mencionada, así como la subida de la carpeta "data/". Dicha carpeta contiene los archivos sobre los cuales trabajé.

Localmente, obtuve la carpeta "data/" al ejecutar:

git clone https://github.com/naaairol/IDSD422-lab1-lectura-datos.git

Posteriormente, subí dicho directorio hacia el ambiente de Google Colab para proceder con la práctica. Es necesario aclarar que en ningún momento tuve que instalar ninguna librería para el ambiente de Python. Afortunadamente las librerías que utilicé estaban ya incluidas, por lo que, únicamente era necesario importarlas

3.3. Lectura de los reportes

3.3.1. Lectura CSV

Objetivamente hablando, fue la más fácil de realizar. Todo es "directo", por decirlo de alguna forma. Únicamente bastó con realizar el código visto a continuación:

```
ventas = pd.read_csv("data/ventas.csv")
```

3.3.2. Lectura JSON

Hay que tomar en cuenta que JSON puede, y tiene, estructuras jerárquicas. Simplemente no es posible darle el mismo tratamiento que a un CSV. Es por esto por lo que, además de cargarlo, indicando la codificación, también es necesario normalizarlo. Para que quede "aplanado". Esto lo realicé mediante el siguiente código:

```
#Abrir el json

with open("data/ventas.json", encoding = "utf-8") as f:

  datos = json.load(f)

#Normalizar

ventas = pd.json_normalize(datos)
```

3.3.3. Lectura XLSX

La lectura resultó bastante más simple que un JSON. Para el caso del XLSX, fue necesario especificar el nombre de la hoja de la cual queremos volcar la información. Esto mediante el parámetro "sheet_name". Esto lo realicé con el código visto a continuación:

```
ventas = pd.read_excel("data/ventas.xlsx", sheet_name="ventas")
```

3.4. Inspección de dataframes y tipos de datos

Una vez que cargué los datos y los dejé en un dataframe, sobre el cual es bastante simple comprobar el tipo de cada columna, así como la dimensión del mismo, bastó con ejecutar el siguiente código sobre cada dataframe creado:

```
print(ventas.shape)

print(ventas.dtypes)
```

Seguidamente tuve que contrastar la salida en consola para cada uno de los tres dataframes. De forma que, pude identificar que, tanto las dimensiones de cada uno, así como el tipo de variable descrito, era exactamente el mismo.

3.5. Indicadores para la empresa

Una vez que se logró todo lo anteriormente planteado, en este paso ya es posible realizar análisis sobre los datos. Lo primero que averigüé sobre los datos fue el

costo promedio de un ticket, obteniendo el máximo de una sucursal de la siguiente manera:

```
ventas["ticket"] = ventas["ventas"]/ventas["clientes"]

print(ventas["ventas"].max())
```

Resultando que el valor promedio del ticket es de 38.07 unidades monetarias.

El siguiente de los descubrimientos que realicé fue el de la sucursal con el mayor número de ventas, así como la sucursal con el menor número. Obteniendo estos resultados tras la ejecución del siguiente código:

```
print("Sucursal con mayor ventas:", ventas.loc[ventas["ventas"].idxmax(), "sucursal"])

print("Sucursal con menor ventas:", ventas.loc[ventas["ventas"].idxmin(), "sucursal"])
```

Siendo estos datos que, la sucursal con mayor número de ventas es Guayaquil Centro y la sucursal con el menor número es Cuenca.

3.6. Preguntas complementarias

- **¿Cuántas filas y columnas tiene la tabla en cada uno de los tres formatos?**

  La tabla siempre tiene cinco filas y cuatro columnas, sin importar el formato. Pude evidenciar esto al ejecutar:

  ```
  print(ventas.shape)
  print(ventas.dtypes)
  ```

  Con su salida:

  ```
  (5, 4)
  sucursal    object
  region      object
  ventas       int64
  clientes     int64
  dtype: object
  ```

- **¿Fue igual en los tres? ¿Por qué?**

  Sí, fue igual. Esto se debe a que los tres describen la misma información, aunque se encuentra provista por tres distintos sistemas.

- **¿Qué tipo de datos se infirió para ventas y clientes? ¿Qué indicaría que salieran como texto?**

  Tanto para ventas, así como para clientes, inferí un tipo de dato numérico. Específicamente int(64). Esto se pudo evidenciar con la ejecución del código indicado en la pregunta anterior. Y que salieran como texto indicaría que los datos de la columna se encuentran sucios, por ejemplo, podría existir la presencia de símbolos no numéricos.

- **En Python, ¿qué estructura devuelve json.load y por qué hay que "aplanarla"?**

  La ejecución de json.load devuelve un diccionario del tipo clave-valor. Justamente por esto es que es necesario aplanar, para convertir esa estructura de datos anidados en un solo nivel.

- **¿Para qué sirve indicar sheet_name/sheet al leer Excel?**

  Para indicar, de forma exacta, la hoja de la cual queremos volcar los datos. Si no se llega a especificar, Python automáticamente va a volcar la primera hoja.

- **¿Qué diferencia ves entre leer en Python y leer en R (conceptos, no sintaxis)?**

  La diferencia principal que pude notar es la forma en que un "dataframe" está implementado en cada uno de estos lenguajes. Mientras que R es un lenguaje cuyo propósito radica en el análisis de datos, por lo que, entiende de forma natural, el concepto de matriz bidimensional, Python no. Al ser un lenguaje más general, estas integraciones se realizan mediante librerías, como es el caso de pandas; utilizada para la realización de esta práctica.

4. ¿Qué aprendí con esta práctica?

En esta práctica aprendí como cargar, para leer, distintos tipos de archivos hacia Python. Independientemente del formato, es decir, no importa que sean CSV, JSON o XLSX. Siempre existen métodos y formas, claro que, con ciertas consideraciones, que permiten leer estos archivos. Además de funciones que permiten conocer las filas y columnas de un dataframe, así como el tipo de variable que presenta cada columna. Adicionalmente, aprendí que esto último es esencial para contrastar dataframes y que, al menos para esta práctica, es lo que permite determinar si los datos son los mismos o no. Aunque, de todas formas, con el desarrollo de los diferentes scripts planteados, podemos ver que los datos son los mismos al contrastar las respuestas tras ejecutar una misma función sobre los tres dataframes diferentes.

5. Conclusiones

Para terminar, se concluye que es totalmente posible leer distintos tipos de archivos en Python, en este caso probando con CSV, JSON y XLSX. Claro que, es necesario tener presentes los conceptos para no olvidar consideraciones como el tema de normalización al transformar JSON en dataframe.

Finalmente, es posible decir que, tras la ejecución de funciones que permiten conocer las dimensiones de un dataframe y los tipos de datos de sus columnas, los datos de los tres reportes distintos, en realidad, contenían la misma información.

Por último, se obtuvieron varios indicadores comerciales de RetailAndina S.A., destacando principalmente que: su sucursal con el mayor número de ventas es Guayaquil centro, la sucursal con el menor número de ventas es Cuenca y, finalmente, el ticket promedio de compra a nivel nacional es 38.07 unidades monetarias.

6. Bibliografía

Pandas development team. (2026). pandas documentation. https://pandas.pydata.org/docs/

Python Software Foundation. (2026). json — JSON encoder and decoder. https://docs.python.org/3/library/json.html

Suárez, C. (2026). IDSD422-lab1-lectura-datos [Repositorio de GitHub]. GitHub. https://github.com/christian-suarez1979/IDSD422-lab1-lectura-datos

7. Anexos

No se incluye ningún anexo.